



COLLEGE OF ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING

ECHO Chat App

- Version 1.0

GitHub link: https://github.com/Natnahom/Echo_chat_app

Group names:

- Natnahom Asfaw, CEP0084/14
- Nebiyu Ermiyas, CEP008914
- Nebyu Daniel, CEP0090/14

Date: 03/01/2025

Software Requirements Specification (SRS)

Table of Contents

1. Introduction	1
1.1 Purpose	
1.2 Document Conventions	
1.3 Intended Audience and Reading Suggestions	
1.4 Product Scope	
1.5 References	
2. Overall Description	2
2.1 Product Perspective	
2.2 Product Functions	
2.3 User Classes and Characteristics	
2.4 Operating Environment	
2.5 Design and Implementation Constraints	
2.6 Assumptions and Dependencies	
3. System Features	4
3.1 User Authentication	
3.2 Real-Time Messaging	
3.3 User Profile Management	
4. External Interface Requirements	6
4.1 User Interfaces	
4.2 Hardware Interfaces	
4.3 Software Interfaces	
4.4 Communication Interfaces	
5. Non-functional Requirements	7
5.1 Performance Requirements	
5.2 Safety Requirements	
5.3 Security Requirements	
5.4 Software Quality Attributes	
5.5 Business Rules	

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a detailed overview of the software requirements for the "ECHO Chat App" (hereafter referred to as "ECHO" or "the Application"). It describes the project's target audience, user interface, hardware and software requirements, and key functional and non-functional features. The purpose of this document is to define the project's scope and provide a clear blueprint for development and testing.

1.2 Document Conventions

This document follows standard SRS formatting. Requirements are prioritized using the following keywords:

- **Must:** An absolute requirement of the system.
- **Should:** A high-priority requirement that is not absolutely necessary but adds significant value.
- **May:** A requirement that is desirable but less critical.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audiences:

- **Project Supervisor/Teacher:** To evaluate the project's scope, objectives, and adherence to academic standards.
- **Developer:** To serve as a guide for the implementation phase, detailing all necessary features and constraints.
- **Testers:** To understand the system's intended behavior and develop appropriate test cases.

1.4 Product Scope

ECHO is a mobile chat application developed as a AASTU project for Mobile programming and computing course. Its primary objective is to demonstrate the practical implementation of real-time communication technologies using the Flutter framework and Firebase backend services. The system will allow users to register, authenticate, and exchange text messages in real-time. It focuses on core chat functionalities, user management, and a simple, intuitive user interface.

1.5 References

- Flutter Documentation: <https://docs.flutter.dev/>
- Firebase Documentation: <https://firebase.google.com/docs>
- Dart Programming Language Specification: <https://dart.dev/guides/language/spec>

2. Overall Description

2.1 Product Perspective

ECHO is a new, self-contained mobile application built from the ground up for this project. It is not a component of a larger system. It relies entirely on external, third-party services for its backend operations:

- **Firebase Authentication:** For handling user account creation, login, and secure session management.
- **Cloud Firestore:** As the primary database for storing all user data (profiles) and chat messages.

2.2 Product Functions

The main functions of the ECHO app are:

- **User Registration & Login:** New users can create an account using an email and password. Existing users can log in securely.
- **Real-Time Chat:** Users can see a list of other registered users and start a conversation. Messages sent by one user appear instantly on the recipient's screen.
- **Chat History:** The app will persistently store and display the conversation history between two users.
- **Unread Message Indication:** The system will show an indicator for conversations that contain unread messages.
- **User Profile Management:** Users can view their own profile information (username, email) and edit their username.
- **Logout:** Users can securely log out of their account from within the app.

2.3 User Classes and Characteristics

There is only one class of user for this application:

- **Registered User:** An individual who has created an account. Their primary goal is to communicate with other registered users. They are expected to have basic proficiency in using a smartphone and mobile applications.

2.4 Operating Environment

- **Target Platform:** Mobile devices.
- **Operating System:** Android (minimum API level as defined by the Flutter project) and iOS (minimum version as defined by the Flutter project).
- **Development Environment:** The application will be developed using Flutter and Dart.
- **Backend Environment:** The application will interface with Google's Firebase platform, specifically utilizing Firebase Authentication and Cloud Firestore.

2.5 Design and Implementation Constraints

- **Platform:** The application must be developed using the Flutter framework to ensure cross-platform compatibility (Android & iOS) with a single codebase.
- **Database Structure:** The Firestore database must contain at least two main collections: users and chats, with fields as defined in the project plan.
- **Authentication:** User authentication must be handled exclusively through Firebase Authentication to ensure secure password storage and management.
- **Project Timeline:** The complete application, including design, development, and testing, must be delivered by the college project deadline.

2.6 Assumptions and Dependencies

- **Assumptions:**
 - Users will have a stable internet connection (Wi-Fi or mobile data) to use the app, as real-time functionality depends on it.
 - Users will have a valid email address for registration.
 - The application will be tested on both Android and iOS emulators/simulators and physical devices.
- **Dependencies:**

- The project is dependent on the continued availability and free-tier access of the Firebase platform.
- The project is dependent on the stability and documentation of the Flutter and Dart SDKs, as well as the Firebase Flutter plugins.

3. System Features

3.1 User Authentication

- **Description:** This feature manages user identity, allowing new users to join and existing users to access the application securely.
- **Stimulus/Response Sequence:**
 1. **Registration:** User enters email and password -> System validates input and creates account via Firebase Auth -> System creates a corresponding user profile document in Firestore -> User is logged in and navigated to the main app screen.
 2. **Login:** User enters email and password -> System validates credentials with Firebase Auth -> On success, user is granted access and navigated to the main app screen.
- **Functional Requirements:**
 - **FR-1.1 (Must):** The system must provide a registration interface for new users to create an account with an email and password.
 - **FR-1.2 (Must):** The system must validate that the email address is in a correct format and that the password meets a minimum length requirement (e.g., 6 characters).
 - **FR-1.3 (Must):** The system must securely store user passwords using Firebase Authentication.
 - **FR-1.4 (Must):** The system must provide a login interface for existing users to authenticate with their email and password.
 - **FR-1.5 (Must):** The system must provide feedback to the user on successful or failed authentication attempts (e.g., "Invalid email or password").

3.2 Real-Time Messaging

- **Description:** This is the core feature, enabling users to send and receive text messages instantly.
- **Stimulus/Response Sequence:**
 1. **Sending a Message:** User types a message and presses send -> System creates a message document in the appropriate Firestore chat subcollection -> The message is instantly displayed in the chat UI.
 2. **Receiving a Message:** A message is written to Firestore by another user -> The app, which is listening for changes, detects the new document -> The new message is automatically fetched and displayed in the UI.
- **Functional Requirements:**
 - **FR-2.1 (Must):** The system must display a list of all registered users, excluding the currently logged-in user.
 - **FR-2.2 (Must):** Tapping on a user from the list must open a chat interface specific to that user.
 - **FR-2.3 (Must):** The system must allow users to send and receive text messages within the chat interface.
 - **FR-2.4 (Must):** The chat interface must display the history of previous messages between the two users.
 - **FR-2.5 (Must):** New messages must appear in the chat interface in real-time without requiring the user to manually refresh the screen.
 - **FR-2.6 (Should):** The system should visually indicate which conversations contain unread messages (e.g., a bold username, a colored dot).

3.3 User Profile Management

- **Description:** This feature allows users to view and manage their own account information.
- **Stimulus/Response Sequence:**
 1. **View Profile:** User navigates to a profile/menu section -> System fetches and displays the user's current email and username from Firestore.

2. **Edit Profile:** User chooses to edit their profile, changes their username, and saves
-> System updates the corresponding user document in Firestore -> System confirms the update was successful.
- **Functional Requirements:**
 - **FR-3.1 (Must):** The system must provide a screen or menu where the logged-in user can view their own profile information (email and username).
 - **FR-3.2 (Should):** The system should provide a feature for the user to edit their username.
 - **FR-3.3 (Must):** The system must provide a clearly labeled "Log Out" button.
 - **FR-3.4 (Must):** When the user taps "Log Out", the system must sign them out via Firebase Authentication and navigate them back to the login screen.

4. External Interface Requirements

4.1 User Interfaces

- The user interface must be built using Flutter's standard widget library.
- The UI must be intuitive and easy to navigate for a typical smartphone user.
- Screens to be implemented include: Login Screen, Registration Screen, User List Screen, Individual Chat Screen, and Profile/Menu Screen.
- The design should be clean and functional, suitable for a college project presentation.

4.2 Hardware Interfaces

- The application will primarily interface with the device's network hardware (Wi-Fi, cellular) to send and receive data.
- No specific hardware peripherals (e.g., camera, Bluetooth) are required for the core functionality.

4.3 Software Interfaces

- **Firebase Authentication SDK:** The application will use the official FlutterFire plugin to communicate with Firebase Authentication for user sign-up and login.
- **Cloud Firestore SDK:** The application will use the official FlutterFire plugin to perform all database operations, including listening for real-time updates.

4.4 Communication Interfaces

- All communication between the mobile app and Firebase services will occur over secure HTTPS connections.
- Real-time updates are facilitated by Firestore's built-in listener mechanism, which uses a persistent WebSocket-like connection for efficiency.

5. Non-functional Requirements

5.1 Performance Requirements

- **PER-1:** The app must load the list of users and initial chat messages within 3 seconds on a standard 4G network connection.
- **PER-2:** Sent messages must appear in the chat interface of both the sender and recipient within 2 seconds under normal network conditions.
- **PER-3:** The application UI must remain responsive (no "Application Not Responding" errors) at all times during network calls.

5.2 Safety Requirements

- **SAF-1:** The application must handle the loss of internet connectivity gracefully, informing the user of the connection issue and attempting to reconnect automatically when the network is restored.
- **SAF-2:** The application must not crash if it receives invalid or unexpected data from the Firestore database.

5.3 Security Requirements

- **SEC-1:** All network communication between the app and Firebase must be encrypted using TLS/SSL.
- **SEC-2:** User passwords must never be stored locally on the device or transmitted in plain text; they are handled entirely by Firebase Authentication.
- **SEC-3:** Firestore Security Rules must be configured to ensure users can only read/write data they are authorized for (e.g., a user can only read messages they are a participant in).
- **SEC-4:** User sessions must be managed securely. Logging out should clear any local session data.

5.4 Software Quality Attributes

- **Reliability:** The core messaging function must be reliable, with a high probability that a sent message will be delivered and displayed.
- **Maintainability:** The codebase should be well-structured, commented, and follow Flutter/Dart best practices to allow for easy updates and bug fixes.
- **Portability:** The application must run on both Android and iOS devices without modification to the core codebase, fulfilling Flutter's promise of portability.
- **Usability:** The application should be simple to understand and use, requiring no instructions for a first-time user to perform basic functions.

5.5 Business Rules

- **BR-1:** A user must have a valid account to send or receive messages.
- **BR-2:** A user can only send messages to other registered users of the application.